

- 二次提交改动：补交学习笔记（之前忘了装到docs里面）；对测试集空缺数据补充方式进行调整。

Titanic_submission之绫濑空复活赛

- 提示：本文疑似成分复杂，若学长看不懂某些神秘文字可直接略过并暗骂sb

上一世，我借助禁术.LogisticRegression，短短一天时间就爆肝出了《泰坦尼克号沉船生存预测》（虽然整整一天也并非短时间了），甚至大逆不道地违反了git仓库提交的最高指令，将如此重要的成果用QQ这种低劣的传输方式交给了吾王（出题人）。毫不意外，吾王雷霆大怒，手刀一挥，我当场身首异处。

但是吾王不知道的是，其实我拥有死亡回归的能力，再次睁眼，我仍坐在图书馆三楼相同的卡座，眼前是我自己的笔记本无疑，上面是我刚写好的篡用禁术.LogisticRegression的代码。换成上一世的我，此时应该为它能成功运行并输出准确率而兴高采烈，但现在的我只感到一股深不可测的恐惧。我手忙脚乱地ctrl+A删除，强迫自己冷静下来思考对策。

现在是下午4点，距离我第二天下午一点提交代码只有不到24小时，我要在这不到一天的时间内不使用禁术，重新自主构建模型，在明天下午一点提交正确的代码，才能改变被斩杀的死档结局。但是这又何尝容易呢？？？焦头烂额的我陷入了和菜月昴一样的绝望，直到我偶然看见了吾王对题目的更新：“PyTorch自主建模”。

这无疑给无头苍蝇的我一缕希望的曙光：既然知道用什么模块建模，那至少有了学习的方向。我找到了吾王恩泽于我的pytorch学习资料，只挑了讲授指令的选集开二倍速严肃观看。

但即便如此，我仅剩的时间仍不足以支撑我完整学习这些细碎的知识。正当我又陷入了和菜月昴一样的绝望时，我偶然看到了电脑桌面上的豆包。既然我保留了上一世的记忆，我也就清楚地知道模型构建、训练、测试与绘图的全流程，我为什么不能告诉豆包我想实现什么指令，对应pytorch里的哪一个函数，哪一套模板？

突然，耳边响起上一世我的怨灵的阴森声音：桀桀桀，你这样做定然又会被吾王斩杀的！桀桀桀.....

但我深信这种方法是带领我逃离死档的正确途径：在《终极试炼》中，我已经系统学习了神经网络模型、Sigmoid激活函数等与这道题紧密相关的内容，我需要的只是一种实现每一步骤的计算机语言。可能目前我还没有完全掌握某一模块的所有指令，但不妨碍我用这个模块完成目标任务，同时也熟悉了这些函数、指令的用法。

“为什么不更自信一点呢？”菜月昴的名言犹在耳畔。我打开了IDLE。

绫濑空的复活赛，打响了。

正文

1.数据处理

- 第一步还是随机种子SEED的设定。和上一世不同的是，这一次还要设定PyTorch内部的随机数序列
- 依旧使用pandas读取csv文件
- 从表格中分离出特征X（扔掉“survived”后的其他标签）和是否存活的标签y，使用copy()是做备份为了避免后续做数据处理污染原数据。
- 划分训练集和测试集的方法和上一世是一样的：train_test_split，这里测试集比例设定的是0.2

- 接下来是对数据预处理：首先空缺的年龄和港口分别由中位数和众数填充（上一世年龄用的是均值想了想还是不如中位数对极端值稳定）
- 接下来是类别特征的转换：sex很简单，男0女1就好（我没有在宣扬什么）但是embarked有三种，上次也说了不能直接赋值会导致自带顺序，需要独热编码。但是我没有用上次笔记里说的get_dummies，因为get_dummies在遇到输入维数并非预期维数时会报错（虽然这里应该不会因为空缺都被补齐了），手搓还是会更保险一点而且也不复杂
- 创建特征表：上次笔记也分析了哪些特征保留哪些特征不要，那种长串字符而且明显和s不s没关系的直接扔了（ID、name），大量空缺的特征也不要了因为没有参考价值
- 上一世没有进行标准化，因为没有引入StandardScaler，但我估计这套数据不标准化也没有大问题，毕竟两次准确率差别也不大。用scaler.fit标准化需要注意一件事：要不要对测试集标准化？如果仅对这道题来说，测试集和训练集是自己划分的，所有数据都是可见的，要不要标准化都无所谓；但是在实际情况中，为防止数据泄露，拿来测试的数据绝对都是“保留食材本味”的，才能起到最好的测试效果。所以此处为了严谨，选择只对训练集标准化（防止过拟合）不对测试集标准化（维持数据机密状态）所以这里有个if-else判断是训练集还是测试集，如果是训练集就标准化，并保存均值、标准差传给测试集；如果是测试集就只接收训练集来的参数，自己不会再被计算出参数
- （二次提交补充）因此，上面填补空缺数据时，为防止数据泄露，也不能用测试集自己的中位数和众数填充；而是用训练集得到的参数填补测试集空缺

2.用PyTorch建模

- 自定义PyTorch数据集 模型不能直接接收numpy/pandas数组，必须转换成tensor（张量），所以用torch.tensor()将原始数据转换成张量格式
- DataLoader读取 上一步已经全部转换成张量，允许使用DataLoader批量读取。设定batch_size为单批次抽取的样本数，分别从训练集和测试集读取。需要注意的是，训练集的样本要打乱（shuffle=True），防止模型记住先后顺序学到虚假规律；而测试集则不打乱（shuffle=False），也是维持数据原样
- torch.nn.Module定义二分类模型 这一部分基本上是对着教程照猫画虎，宏观层面就是先创建一个全连接线性层（神经网络的单层），然后前向计算；原理层面说实话也没太搞明白（笑哭）。损失的计算使用自带Sigmoid的nn.BCEWithLogitsLoss()（二分类损失函数）。此处笨人在豆包建议下引入了《终极试炼》问到过的Adam优化器，属于是学以致用了

3.主体部分

- 终于到了喜闻乐见的训练模型的阶段了（乐）设定轮次为600，然后请出模型训练三巨头：
`optimizer.zero_grad()` #梯度清零 `loss.backward()` #反向传播 `optimizer.step()` #更新参数
- 分别在训练模式和测试模式使用Sigmoid函数预测，统计猜对猜错数并累加到总数中，在每一轮最后计算整轮平均损失和准确率。当轮次到50的倍数时，输出结果显示该轮的损失和准确率
- 最后依据累加得到的总样本数、总正确数、总错误数和计算得到的准确率，进行最终结果输出。主体部分大成功！

4.绘图

- 绘制折线图的代码和夏令营题一模一样，不必再赘述。但须注意的是，绘图的数据来自前面我们定义的loss和acc的history数组，在每一轮末尾要记得append一下该轮的数据

旅路は続く

我吸取了上一世的教训，开始写日记，这篇笔记就是写日记第一天的内容。

人是不断进步的，但有的时候，你会从这些过去的你写的文字中学到不一样的东西。

请你记住，绫濑空：你做的每一道题都不会白做，你花的每一分钟都不会白费；你写的每一篇题解都有亮眼之处，你敲下的每一行代码都是成长的印记。你曾经经历的一切苦难都化作你心中的沟壑，你曾经留下的所有伤痕都成为你涅槃的徽章。

或许你还能期待，一个穿越回来的老头，拿着一本日记，自称是未来的你，向你指点迷津。

所以，You never walk alone.

By the way，请关注本周无职转生最新一集《日记》喵！

绫濑空 2026年9月